CYPRUS INTERNATIONAL UNIVERSITY

CMPE411 Systems Programming

CHAPTER 1 - Java File I/O (Weeks 1 - 2)

Assoc.Prof.Dr. Tamer TULGAR

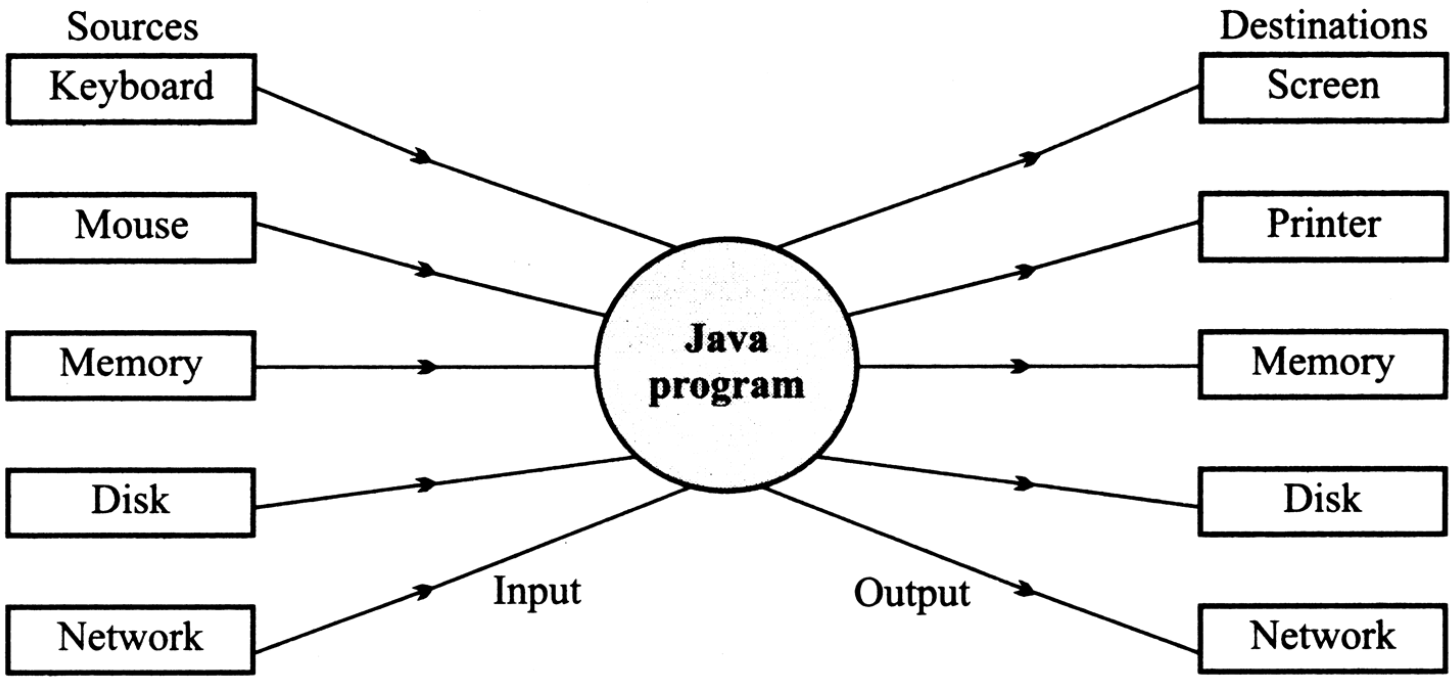
Spring 2019-2020

File Processing

- Storing and manipulating data using files is known as file processing.
- Reading/Writing of data in a file can be performed at the level of bytes, characters, or fields depending on application requirements.
- Java also provides capabilities to read and write class objects directly.
- The process of reading and writing objects is called object serialisation.

I/O and Data Movement

- The flow of data into a program (input) may come from different devices such as keyboard, mouse, memory, disk, network, or another program.
- The flow of data out of a program (output) may go to the screen, printer, memory, disk, network, another program.
- Both input and output share a certain common property such as unidirectional movement of data – a sequence of bytes and characters and support to the sequential access to the data.



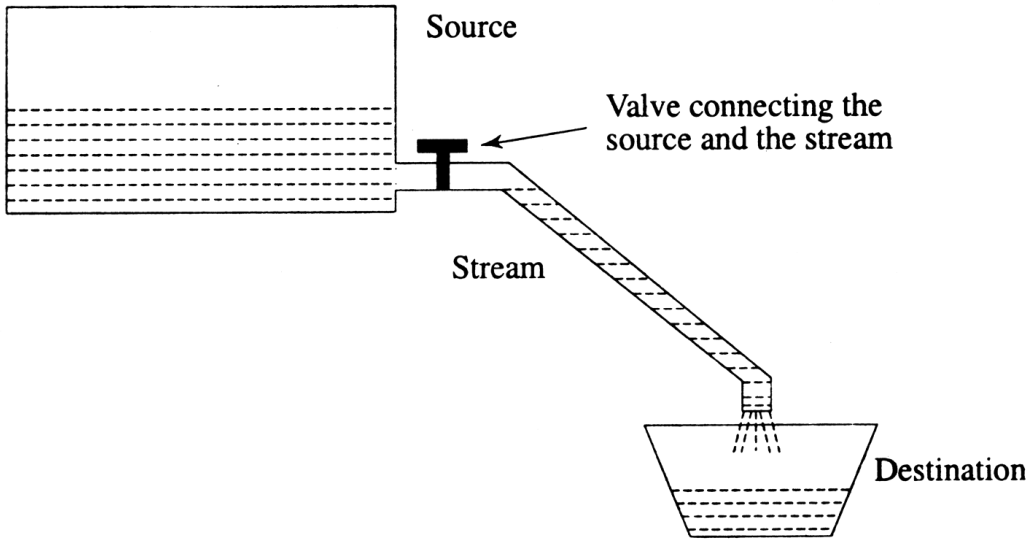
Relationship of Java program with I/O devices

Streams

- Java Uses the concept of Streams to represent the ordered sequence of data, a common characteristic shared by all I/O devices.

- Streams presents a uniform, easy to use, object oriented interface between the program and I/O devices.

- A stream in Java is a path along which data flows (like a river or pipe along which water flows).



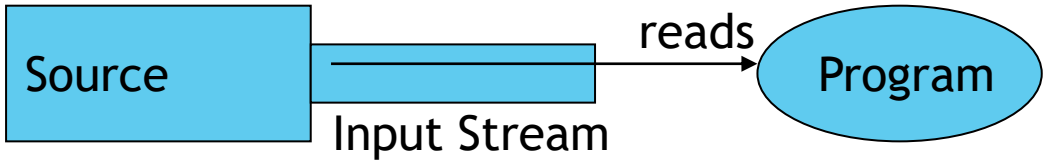
Conceptual view of a stream

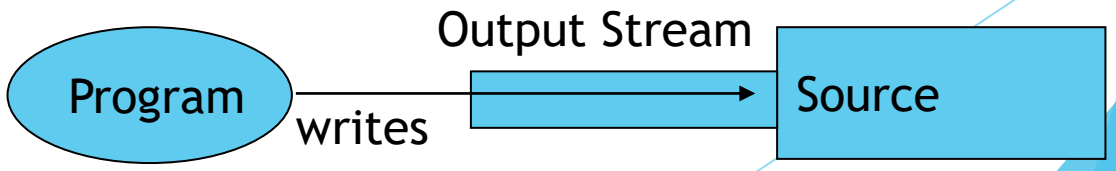
Stream Types

- The concepts of sending data from one stream to another (like a pipe feeding into another pipe) has made streams powerful tool for file processing.

- Connecting streams can also act as filters.

- Streams are classified into two basic types:
  - **Input Steam**
  - **Output Stream**



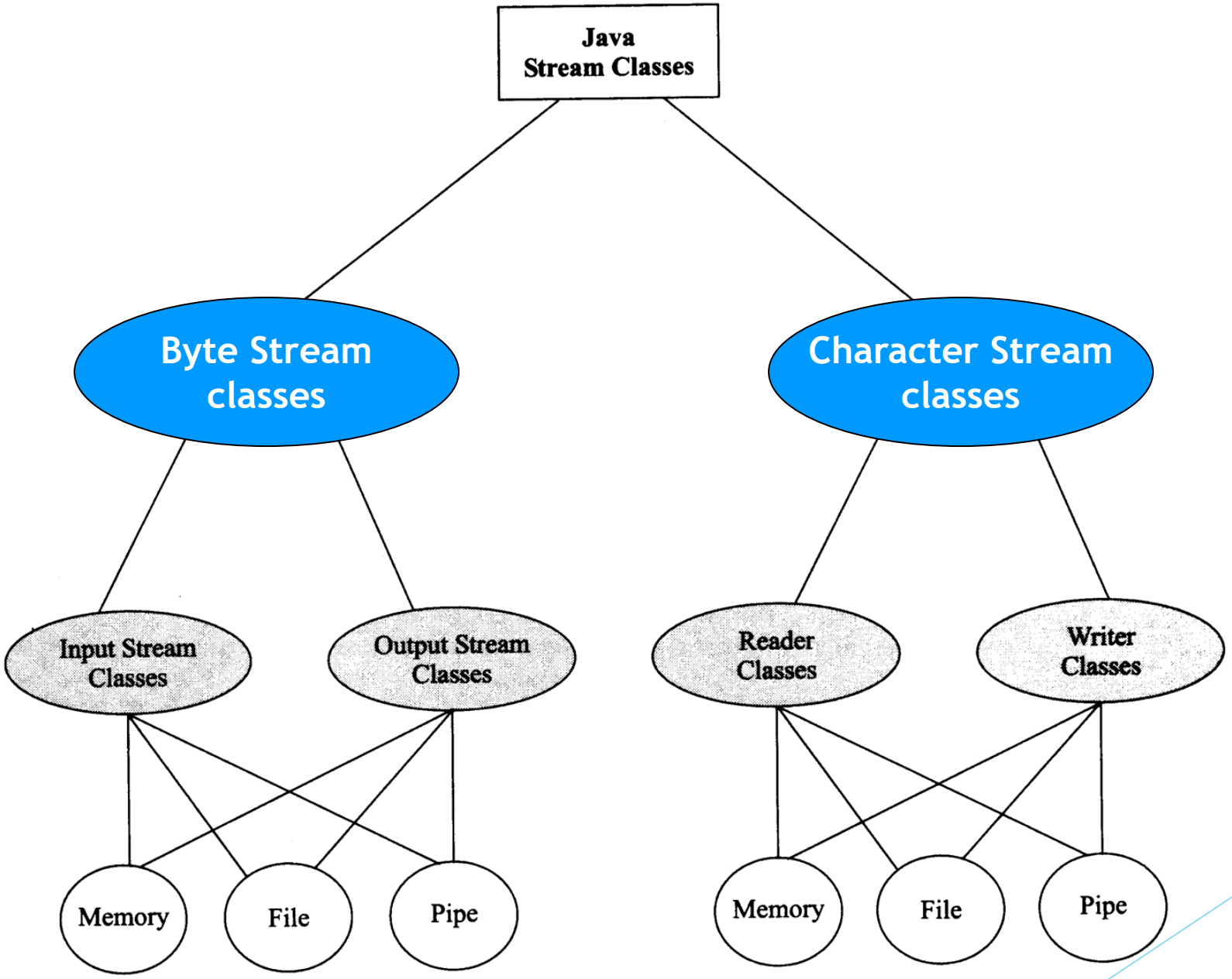


Java Stream Classes

- Input/Output related classes are defined in **java.io** package.
- Input/Output in Java is defined in terms of streams.
- A *stream* is a sequence of data, of no particular length.
- Java I/O classes can be categorised into two groups based on the data type one which they operate:
  - ***Byte streams***
  - ***Character Streams***

Byte Streams	Character streams
Operated on 8 bit (1 byte) data.	Operates on 16-bit (2 byte) unicode characters.
Input streams / Output streams	Readers / Writers

Classification of Java Stream Classes



Java.io.File class

- File class is a general machine independent interface to the file system.

- File class is not for the contents of a file, but the file object

  Directories are File Objects in java

- Java's File class is an abstract representation of file and directory pathnames.

  So, in general: if you *only* need to pass a file name when opening a file, then a flat string does fine. But as soon as it might be helpful to do *other things* with that file, then using the **abstraction** is the way to go.

Java.io.File methods

- getName()
- getPath()
- getAbsolutePath()
- getParent()
- isAbsolute()
- lastModified()
- IsFile()
- isDirectory()
- canRead()
- canWrite()

Java.io.File class

- A sample program to read all files in a directory recursively and display them on the monitor.

```java
private static void getAllFiles(File curDir)
{
    File[] filesList = curDir.listFiles();
    for(File f : filesList)
    {
        if (f.isDirectory())
            getAllFiles(f);
        if (f.isFile())
            System.out.println(f.getName());
    }
}

public static void main(String[] args) {

    File curDir = new File(".");
    getAllFiles(curDir);
}
```

- In this case, the output is the project folder contents of the IntelliJ project folder.

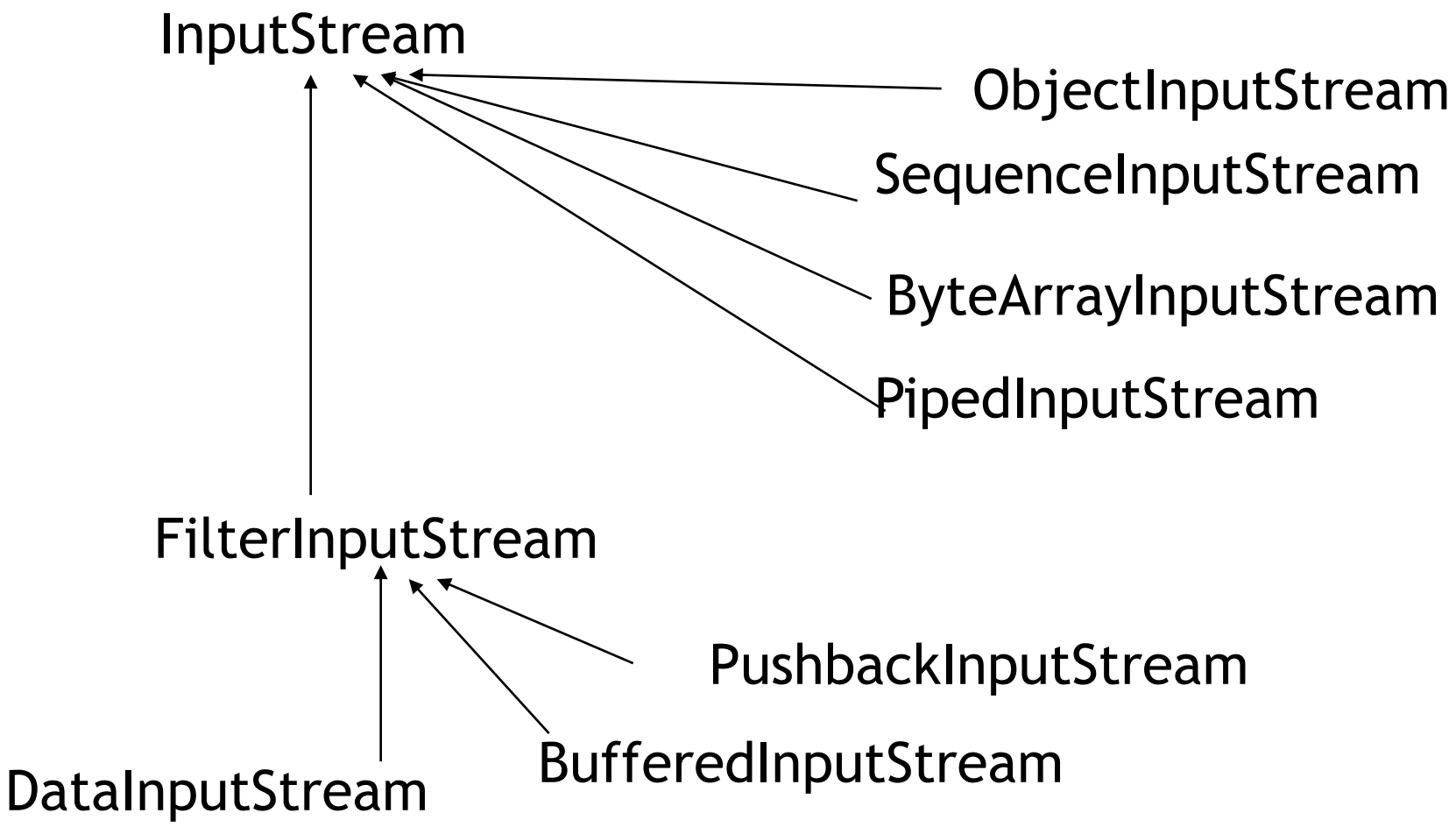
```
Project_Default.xml
description.html
misc.xml
modules.xml
project-template.xml
workspace.xml
encodings.xml
uiDesigner.xml
.gitignore
Main.java
InteliJ Test.kotlin_module
Main.class
Main - Copy.class
toshiba registration.png
test6.png
test7.png
Project_Default.xml
description.html
encodings.xml
misc.xml
modules.xml
project-template.xml
uiDesigner.xml
workspace.xml
Main.class
OptimizedRandomAccessFile.class
.project
InteliJ Test.iml
InteliJ Test.iml
.project
```

BYTE INPUT/OUTPUT

Reading/Writing Bytes

- As most file systems use only 8-bit bytes, Java supports number of classes that can handle bytes. The two most commonly used classes for handling bytes are:
  - **FileInputStream**
  - **FileOutputStream**

Byte Input Streams



Byte Input Streams - operations

public abstract int read()	Reads a byte and returns as a integer 0-255
public int read(byte[] buf, int offset, int count)	Reads and stores the bytes in buf starting at offset. Count is the maximum read.
public int read(byte[] buf)	Same as previous offset=0 and length=buf.length()
public long skip(long count)	Skips count bytes.
public int available()	Returns the number of bytes that can be read.
public void close()	Closes stream

Byte Input Stream - example

- Count total number of bytes in the file

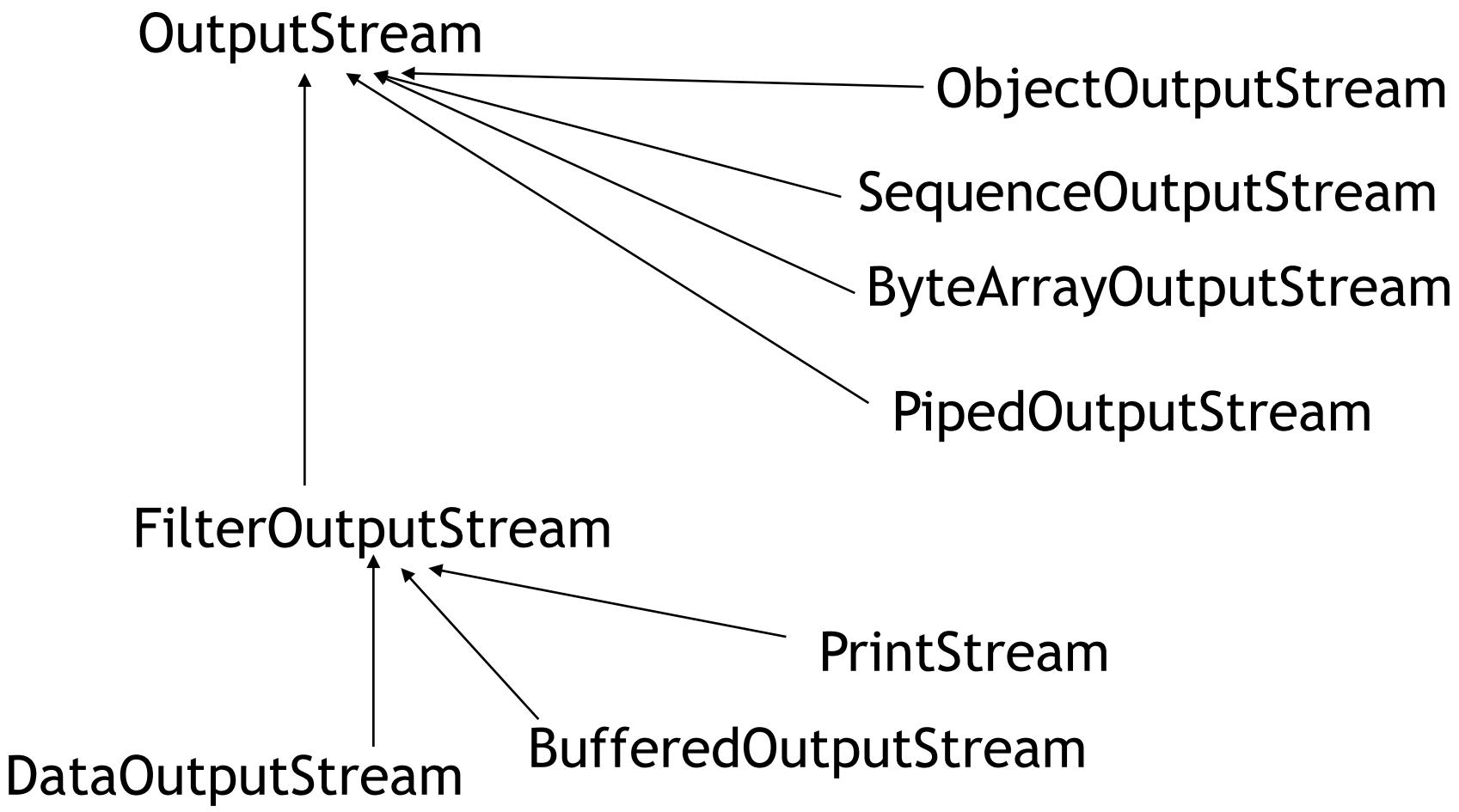
```
import java.io.*;

class CountBytes {
	public static void main(String[] args) throws FileNotFoundException, IOException
	{
		FileInputStream in;
		in  =  new FileInputStream("InFile.txt");

		int total = 0;
		while (in.read() != -1)
			total++;
		System.out.println(total + " bytes");

		in.close();
	}
}
```

What happens if the file did not exist

- JVM throws exception and terminates the program since there is no exception handler defined.

```
Exception in thread "main" java.io.FileNotFoundException: FileIn.txt (No such file or directory)
    at java.io.FileInputStream.open(Native Method)
    at java.io.FileInputStream.<init>(FileInputStream.java:64)
    at CountBytes.main(CountBytes.java:12)
```

Byte Output Streams



Byte Output Streams - operations

public abstract void write(int b)	Write *b* as bytes.
public void write(byte[] buf, int offset, int count)	Write *count* bytes starting from *offset* in *buf*.
public void write(byte[] buf)	Same as previous *offset=0* and *count = buf.length()*
public void flush()	Flushes the stream.
public void close()	Closes stream

FileOutputStream - Example

```
public class WriteBytes {
        public static void main (String[] args)
        {
              byte cities[] = {'M', 'e', 'l', 'b', 'o', 'u', 'r', 'n', 'e',
                                                '\n', 'S', 'y','d', 'n', 'e', 'y', '\n' };

                FileOutputStream outFile;
                try{
                        outFile = new FileOutputStream("City.txt");
                        outFile.write(cities);
                        outFile.close();
                }
                catch(IOException e)
                {
                        System.out.println(e);
                        System.exit(-1);
                }
        }
}
```

Pay attention that if City.txt <u>already exists</u>, then it will be <u>overwritten</u>!!!

If not, a new file will be created

Files and Exceptions

- When creating files and performing I/O operations on them, the systems generates errors.

The basic I/O related exception classes are given below:

- EOFException – signals that end of the file is reached unexpectedly during input.
- **FileNotFoundException** – file could not be opened
- InterruptedIOException – I/O operations have been interrupted
- **IOException** – signals that I/O exception of some sort has occurred – very general I/O exception.

Syntax

- Each I/O statement or a group of I/O statements much have an exception handler around it/them as follows:

```
try {
…// I/O statements – open file, read, etc.
}
catch(IOException e) // or specific type exception
{
    …//message output statements
}
```

Example

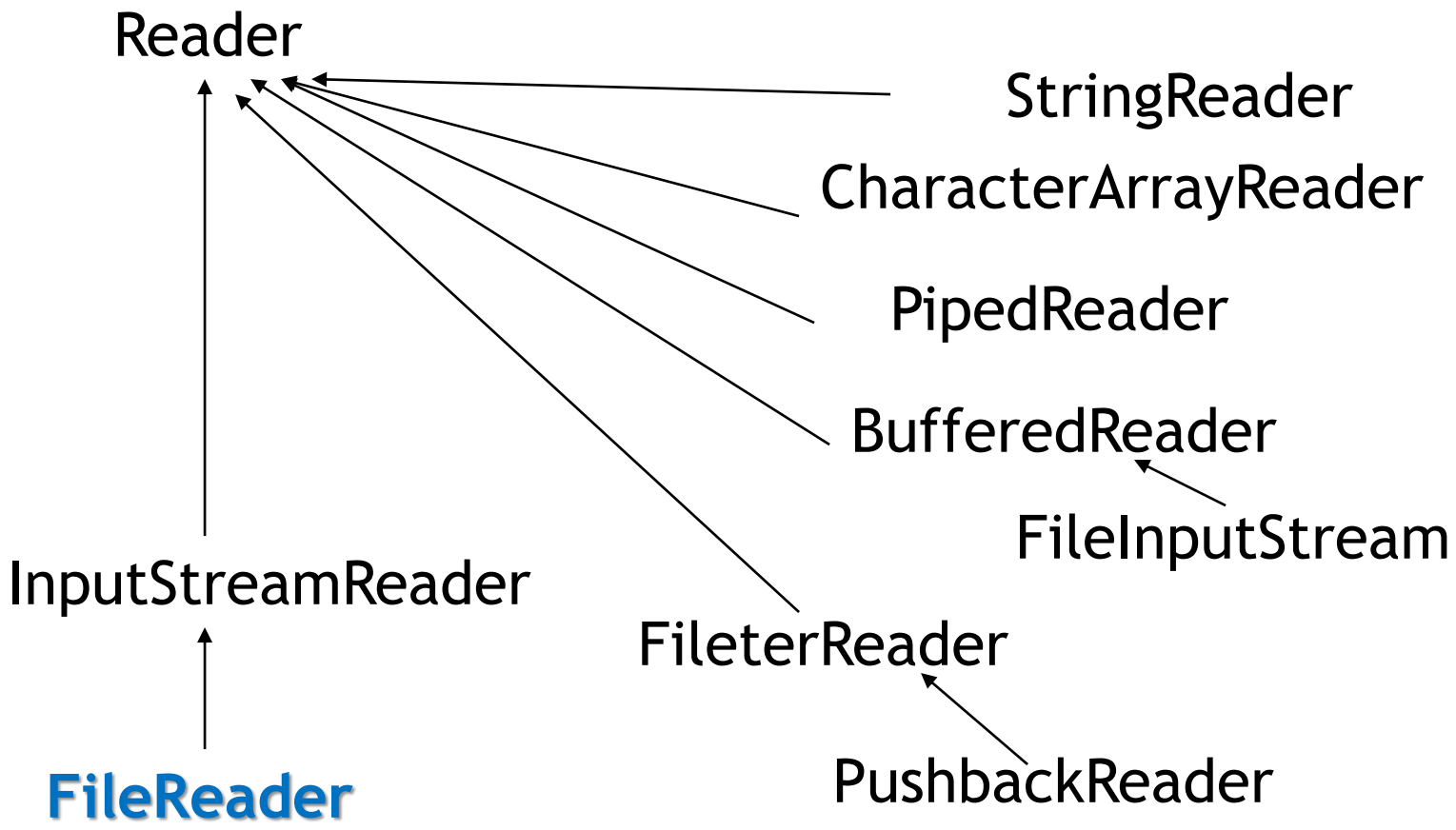
```
import java.io.*;

class CountBytesNew {


        public static void main (String[] args) throws FileNotFoundException, IOException
        // throws is optional in this case
        {

                FileInputStream in;

                try{

                        in = new FileInputStream("FileIn.txt");

                        int total = 0;

                        while (in.read() != -1)

                                total++;

                        System.out.println("Total = " + total);

                        in.close();

                }

                catch(FileNotFoundException e1)

                {

                        System.out.println("FileIn.txt does not exist!");

                }

                catch(IOException e2)

                {

                        System.out.println("Error occured while read file FileIn.txt");

                }

        }

}
```

CHARACTER INPUT/OUTPUT

Reading and Writing Characters

- As pointed out earlier, subclasses of Reader and Writer implement streams that can handle characters.
- The two subclasses used for handling **characters** in file are:
  - **FileReader**
  - **FileWriter**
- The FileReader and FileWriter classes are used to read and write 16-bit unicode characters.
- Remember, while opening a file, we can pass either file name or File object during the creation of objects of the above classes.

Reader Class Hierarchy



Reader - operations

public int read()	Reads a character and returns as a integer 0-255
public int read(char[] buf, int offset, int count)	Reads and stores the characters in buf starting at offset. count is the maximum read.
public int read(char[] buf)	Same as previous offset=0 and length=buf.length()
public long skip(long count)	Skips count characters.
public boolean()	Returns true if the stream is ready to be read.
public void close()	Closes stream

Reader - example

- Count total number of spaces in the file

```
import java.io.*;

public class CountSpace {
	public static void main (String[] args)
		throws IOException
	{
		FileReader in;
		in = new FileReader("FileIn.txt");
		int ch, total, spaces;

		spaces = 0;

		for (total = 0 ; (ch = in.read()) != -1; total++)
		{
			if(Character.isWhitespace((char) ch))
				spaces++;
		}
		System.out.println(total + " chars " + spaces + " spaces ");
		in.close();
	}
}
```

Reading a file (by providing a string path)

Providing directly the filename

```java
//Read File using FileReader char by character
String filename = "d:\\test.txt";

FileReader readfile = null;

try {
    readfile = new FileReader(filename);

    int data;
    while ((data = readfile.read()) != -1)
        System.out.print((char) data);

} catch (FileNotFoundException e) {
    e.printStackTrace();
} catch (IOException e) {
    e.printStackTrace();
} finally {
    try {
        readfile.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

- You are only providing a path as a plain String
- Hence, you do not have any abilities to check anything related to the file system i.e. if file exists or not, its full path etc.

Pay attention that FileReader always returns an Integer !!!

To see the read characters, you will have to type cast int to char

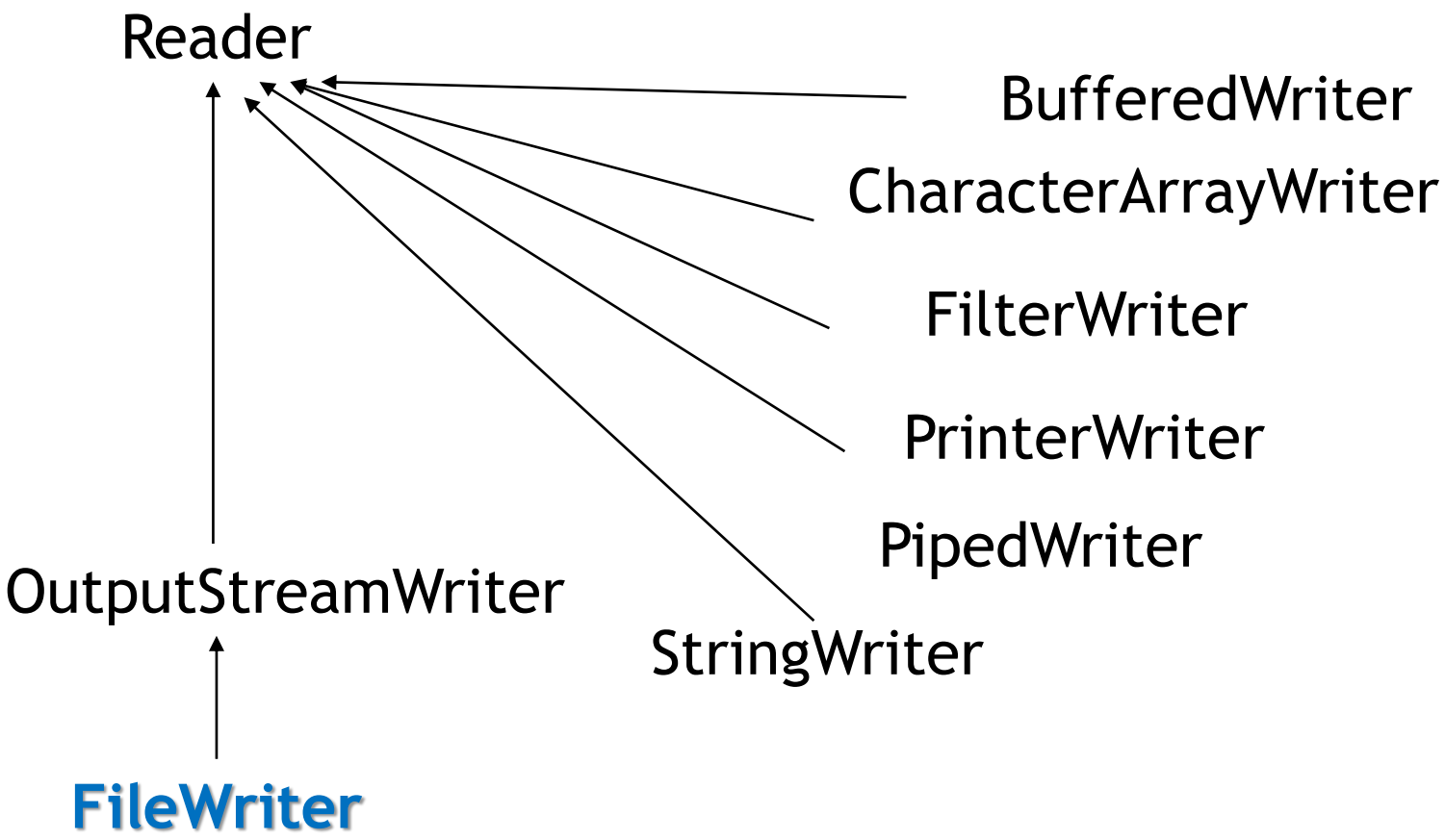
Reading a file (by providing a File object)

Providing a File object

```java
//Read File using FileReader char by character
String filename = "d:\\testfile.txt";

File rdfl = new File(filename);

if(rdfl.exists())
{
    FileReader readfile = null;

    try {
        readfile = new FileReader(rdfl);

        int data;
        while ((data = readfile.read()) != -1)
            System.out.print((char) data);

    } catch (FileNotFoundException e) {
        e.printStackTrace();
    } catch (IOException e) {
        e.printStackTrace();
    } finally {
        try {
            readfile.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
else
    System.out.println("The file you are trying to open " + rdfl.getAbsolutePath()+" does not exist !!!! ");
```

- In this case, you are abstracting your file as a File object.
- For example, you may check if your file exists before attempting to open your file.
- You will have filesystem related abilities for your file now!!!

Writer Class Hierarchy



Writer - operations

public abstract void write(int ch)	Write ch as characters.
public void write(char[] buf, int offset, int count)	Write count characters starting from offset in buf.
public void write(char[] buf)	Same as previous offset=0 and count = buf.length()
public void write(String str, int offset, int count)	Write count characters starting at offset of str.
public void flush()	Flushes the stream.
public void close()	Closes stream

FileWriter Example

```java
File file = new File("d:\\test_wr.txt");
FileWriter fr = null;
try {
    fr = new FileWriter(file);
    fr.write('A');
    fr.write(67);
    fr.write("456");
} catch (IOException e) {
    e.printStackTrace();
} finally {
    //close resources
    try {
        fr.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

test_wr.txt

```
AC456
```

FileWriter is a character stream writer !!!

Hence, the integer 67 in `fr.write(67);` will be treated as an ASCII code !!!

Pay attention that there is no implicit new line written by the write() method

if test_wr.txt <u>already exists</u>, then it will be <u>overwritten</u>!!!

If not, a new file will be created

BUFFERED INPUT/OUTPUT

Buffered Streams

- Java supports creation of buffers to store temporarily data that read from or written to a stream. This process is known as *buffered I/O* operation.

- Buffered stream classes – BufferedInputStream, BufferedOutputStream, **BufferedReader, BufferedWriter** buffer data to avoid every read or write going to the stream. **(Default buffer size = 8KB)**

- **These are used in file operations since accessing the disk for every character read is not efficient.**

Buffered Streams

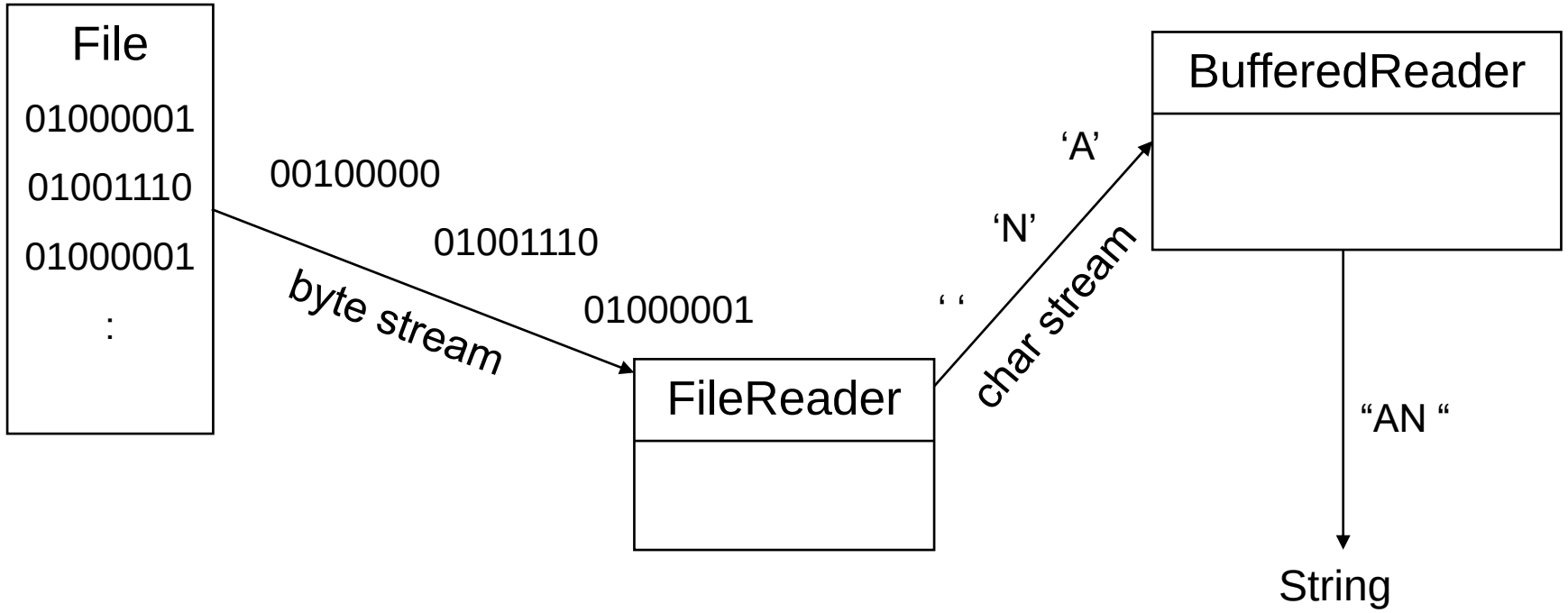
- Buffered character streams understand lines of text.
- BufferedWriter has a newLine method which writes a new line character to the stream.
- BufferedReader has a readLine method to read a line of text as a String.
- Buffered Streams are preferable where better **optimized I/O performance is needed.**

BufferedReader - example

- Use a BufferedReader to read a file **one line at a time** and print the lines to standard output

```
import java.io.*;

class ReadTextFile {
	public static void main(String[] args) throws FileNotFoundException, IOException
	{
		BufferedReader in;
		in = new BufferedReader(new FileReader("Command.txt"));

		String line;
		while (( line = in.readLine())  !=  null )
			System.out.println(line);


		in.close();
	}
}
```

BufferedReader



- Wrapping a BufferedReader around a FileReader will result in getting the file contents as a complete string directly. (Very Efficient I/O !!!)

- As a result you will have to string process what you get to achieve data fields that have different data types

BufferedReader - example

```java
String filename = "d:\\test.txt";
FileReader readfile = null;
BufferedReader readstream = null;

try {

    readfile = new FileReader(filename);
    readstream = new BufferedReader(readfile);

    String line;

    while ((line = readstream.readLine()) != null)
        System.out.println(line);


} catch (FileNotFoundException e) {
    // TODO Auto-generated catch block
    e.printStackTrace();
} catch (IOException e) {
    // TODO Auto-generated catch block
    e.printStackTrace();
} finally {
    try {
        readstream.close();
        readfile.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

We are Wrapping a BufferedReader around a FileReader

Pay attention that BufferedReader's readLine() always returns a String !!!

Scanner to read files (Another Buffered Input)

```java
//Read File using Scanner (The modern way)
File rdfile = new File( pathname: "d:\\test123.txt");
Scanner fs = null;
try
{
    fs = new Scanner(rdfile);
    while (fs.hasNext()) {
        System.out.println(fs.next());
        System.out.println(fs.nextDouble());
        System.out.println(fs.next());
        System.out.println(fs.nextInt());
    }

} catch (IOException e) {
    e.printStackTrace();
} finally {
    fs.close();
}
```

- Remember your keyboard input use case
- Scanner can read from files as well
- Scanner is a Buffered Input Class
- Like fprintf() in C language, you can directly read various data types using next(), nextInt() etc. functions

test123.txt

```
john 123,2 a 45
tom 23,12 b 23
jane 985,445 c 12
```

```
john
123.2
a
45
tom
23.12
b
23
jane
985.445
c
12
```

Differences Between BufferedReader and Scanner classes

Here are the 5 key differences between the Scanner and BufferedReader class of Java API:

1. A scanner is a much more powerful utility than BufferedReader. It can parse the user input and read an int, short, byte, float, long and double apart from String. On the other hand, BufferedReader can only read String in Java.

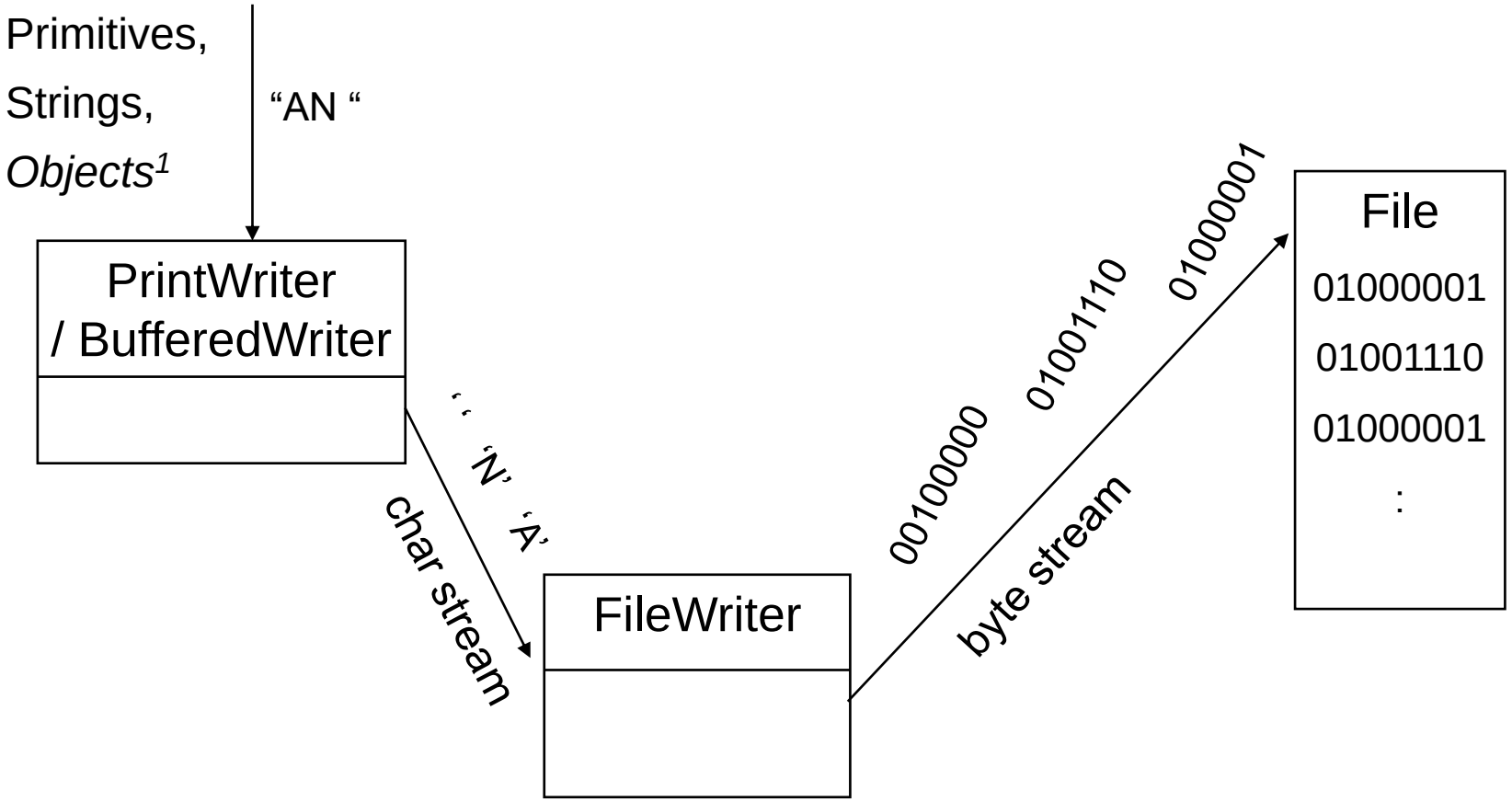
2. BuffredReader has a significantly large buffer (8KB) than Scanner (1KB), which means if you are reading long String from a file, you should use BufferedReader but for short input and input other than String, you can use Scanner class.

3. BufferedReader is older than Scanner. It's present in Java from JDK 1.1 onward but Scanner is only introduced in JDK 1.5 release.

4. Scanner uses regular expression to read and parse text input. It can accept custom delimiter and parse text into primitive data type e.g. int, long, short, float or double using nextInt(), nextLong(), nextShort(), nextFloat(), and nextDouble() methods, while BufferedReader can only read and store String using readLine() method.

5. Another major difference between BufferedReader and Scanner class is that BufferedReader is **synchronized** while Scanner is not. This means, you cannot share Scanner between multiple threads but you can share the BufferedReader object.

BuffereWriters



- Wrapping a BufferedWriter around a FileReader will result in a very efficient I/O !!! Especially when you are directly writing new text files.

BufferedWriter - example

```java
//Write File using Buffered writer
File file2 = new File("d:\\test_wr2.txt");
FileWriter fr2 = null;
BufferedWriter bw = null;
String data2 = "123456";

int noOfLines = 10;
try {
    fr2 = new FileWriter(file2);
    bw = new BufferedWriter(fr2);

    for (int i = noOfLines; i > 0; i--)
        bw.write(data2);

} catch (IOException e) {
    e.printStackTrace();

} finally {
    try {
        bw.close();
        fr2.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

test_wr2.txt

```
123456
123456
123456
123456
123456
123456
123456
123456
123456
123456
```

```java
//Write File using Buffered writer
File file2 = new File("d:\\test_wr2.txt");
FileWriter fr2 = null;
BufferedWriter bw = null;
String data2 = "123456";
String dataWithNewLine = data2 + System.getProperty("line.separator");
int noOfLines = 10;
try {
    fr2 = new FileWriter(file2);
    bw = new BufferedWriter(fr2);

    for (int i = noOfLines; i > 0; i--) {
        bw.write(dataWithNewLine);

    }
} catch (IOException e) {
    e.printStackTrace();

} finally {
    try {
        bw.close();
        fr2.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

test_wr2.txt

```
123456123456123456123456123456123456123456123456123456123456
```

PrintWriter

```java
//Write File using Print writer
File file3 = new File("d:\\test_wr3.txt");
FileWriter fw3 = null;
PrintWriter pw = null;
int data1 = 123;
double data2 = 15.4;
String data3 = "Hello";

try {
    fw3 = new FileWriter(file3);
    pw = new PrintWriter(fw3);

    pw.println(data1);
    pw.print(data2);
    pw.println(data3);
    pw.println();
    pw.printf("%d\t%f\t%s", data1, data2, data3);

} catch (FileNotFoundException e) {
    e.printStackTrace();
} catch (IOException e) {
    e.printStackTrace();
} finally {
    pw.close();
}
```

test_wr3.txt

```
123
15.4Hello

123 15.400000   Hello
```

- PrintWriter uses Buffered Output as well
- The main reason to use the PrintWriter is to get access to the printXXX methods like println().
- Mainly useful if you would like formatted output (like C language printf).
- You can essentially use a PrintWriter to write to a file just like you would use System.out to write to the console.
- A BufferedWriter is an efficient way to write to a file, as it will buffer the characters in Java memory before writing to the file.
- There is no such concept as a "PrintReader"; the closest you will get is probably java.util.Scanner.
- **if test_wr3.txt already exists, then it will be overwritten!!! If not, a new file will be created**

Buffered Byte Streams (Manual Buffering)

- Remember that you can always manually buffer your streams.
- Pay attention once again that, BufferedReader and BufferedWriter are character readers/writers.
- **You can always use read/write methods with byte array (buffer) parameters for byte buffering.**
  Please remember the following methods from previous slides:

  public int read(byte[] buf)

  public int write(byte[] buf)
- You would like to buffer your streams in case you want to increase your byte I/O performance.

Effect of Buffering your Byte Stream → Performance Optimization

```java
String from_filename = args[0];
String to_filename = args[1];

File copy_from = new File(from_filename);
File copy_to = new File(to_filename);

FileInputStream read_from = null;
FileOutputStream write_to = null;

long startTime = System.currentTimeMillis();
try{
    read_from = new FileInputStream(copy_from);

    write_to = new FileOutputStream(copy_to);

    int data;
    while((data = read_from.read() ) != -1)
        write_to.write(data);

} catch (FileNotFoundException e) {
    e.printStackTrace();
} catch (IOException e) {
    e.printStackTrace();
} finally{
    try {
        read_from.close();
        write_to.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
long endTime = System.currentTimeMillis();
long timeElapsed = endTime - startTime;
System.out.println("Execution time in milliseconds: " + timeElapsed);
```

BYTE BY BYTE

Direct read -> write one byte **UNBUFFERED**

java ByteCopy sample.png test.png
Execution time in milliseconds: **5805**

```java
String from_filename = args[0];
String to_filename = args[1];

File copy_from = new File(from_filename);
File copy_to = new File(to_filename);

FileInputStream read_from = null;
FileOutputStream write_to = null;

long startTime = System.currentTimeMillis();
try{
    read_from = new FileInputStream(copy_from);

    write_to = new FileOutputStream(copy_to);

    int data;
    byte[] buffer = new byte[4096];
    while((data = read_from.read(buffer) ) != -1)
        write_to.write(buffer);

} catch (FileNotFoundException e) {
    e.printStackTrace();
} catch (IOException e) {
    e.printStackTrace();
} finally{
    try {
        read_from.close();
        write_to.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
long endTime = System.currentTimeMillis();
long timeElapsed = endTime - startTime;
System.out.println("Execution time in milliseconds: " + timeElapsed);
```

BYTE BY BYTE

With 4KB **Buffered** read -> write

java ByteCopy sample.png test.png
Execution time in milliseconds: **7**

CHECK OUT THE EXECUTION TIMES !!!!

Summary

- Streams provide uniform interface for managing I/O operations in Java irrespective of device types.
- Java supports classes for handling Input Steams and Output steams via java.io package.
- Exceptions supports handling of errors and their propagation during file operations.
- All Java I/O classes are designed to operate with Exceptions.
- Use Exceptions and your own handler with files to mange runtime errors.
- FileInputStream / FileOutputStream classes support bytes-based File I/O.
- Subclasses FileReader / FileWriter support characters-based (2 bytes) File I/O
- Subclasses BufferedReader / BufferedWriter can be used to wrap FileReader / FileWriter classes to provide buffered I/O
- **Buffered operations support efficient I/O operations.**

Recommendations

- When you are reading / writing binary files you have only one way to go !!

  FileInputStream and FileOutputStream

- Always consider buffering your byte stream if you are not reading / writing a single byte ☺

- When you are reading / writing text files (i.e. Unicode characters);

  FileReader and FileWriter wrapped around a Buffered I/O object.

  - Typically;

    Wrapping a BufferedReader around a FileReader

    Wrapping a PrintWriter around a FileWriter

  is practical in most of the cases ☺

- Getting used to providing **a File object rather than a String file path is always a good habit !!!.**

Appending

- If you studied the previous slides carefully, you must have realized that we have not talked about **appending** to a file yet.(we either read a file or created/overwrote a file's contents.)

- Appending : Adding/Modifying data to an existing file, that already contains data.

- Also, you must have realized that we always worked on **sequential** file access.

- Consider the following file output examples:

Appending

```java
//Write File using Print writer
File file3 = new File("d:\\test_wr3.txt");
FileWriter fw3 = null;
PrintWriter pw = null;
int data1 = 123;
double data2 = 15.4;
String data3 = "Hello";

try {
    fw3 = new FileWriter(file3, true);
    pw = new PrintWriter(fw3);

    pw.println(data1);
    pw.print(data2);
    pw.println(data3);
    pw.println();
    pw.printf("%d\t%f\t%s", data1, data2, data3);

} catch (FileNotFoundException e) {
    e.printStackTrace();
} catch (IOException e) {
    e.printStackTrace();
} finally {
    pw.close();
}
```

- The second parameter in the FileWriter constructor overload is the **boolean append** parameter.

FileWriter(File file, boolean append)

- Setting it to true simply means, you want to append to the file.
- Since we are performing **sequential file I/O**, **appending will always be at the end of the file** (i.e. wherever the file pointer shows, in sequential I/O, the end of the file)
- A similar overload that contains the append:true parameter, is also existing in FileOutputStream constructor as well.(Again appending will take place at the end of the file)

FileOutputStream(File file, boolean append)

- If we execute the program two times the output file contents will look like:

```
123
15.4Hello

123 15.400000   Hello123
15.4Hello

123 15.400000   Hello
```

What if we would like to;

1. append at any point in an existing file?

OR

2. read a file starting not from the beginning but from a point we would like and not going till the end?

Answer is;

RANDOM ACCESS FILES

Random Access

- Java allows a program to perform *random file access*.
- In random file access, a program may immediately jump to any location in the file.
- To create and work with random access files in Java, you use the `RandomAccessFile` class.

```
RandomAccessFile(String filename, String mode)
```

- *`filename`*: the name of the file.
- *`mode`*: a string indicating the mode in which you wish to use the file.
  - "r" = reading
  - "rw" = for reading and writing.

Random Access

```
// Open a file for random reading.
RandomAccessFile randomFile = new RandomAccessFile("MyData.dat", "r");

// Open a file for random reading and writing.
RandomAccessFile randomFile = new RandomAccessFile("MyData.dat", "rw");
```

- When opening a file in "r" mode where the file does not exist, a `FileNotFoundException` will be thrown.
- Opening a file in "r" mode and trying to write to it will throw an `IOException`.
- If you open an existing file in "rw" mode, it will not be deleted and the file's existing content will be preserved.

Random Access

- Items in a sequential access file are accessed one after the other.
- Items in a random access file are accessed in any order.
- If you open a file in "rw" mode and the file does not exist, it will be created.
- A file that is opened or created with the `RandomAccessFile` class is treated as **a binary file**.

The File Pointer

- The `RandomAccessFile` class treats a file as a stream of bytes.
- The bytes are numbered:
  - the first byte is byte 0.
  - The last byte's number is one less than the number of bytes in the file.
- These byte numbers are similar to an array's subscripts, and are used to identify locations in the file.
- Internally, the `RandomAccessFile` class keeps a long integer value known as the *file pointer*.

The File Pointer

- The *file pointer* holds the byte number of a location in the file.
- When a file is first opened, the file pointer is set to 0.
- When an item is read from the file, it is read from the byte that the file pointer points to.
- Reading also causes the file pointer to advance to the byte just beyond the item that was read.
- If another item is immediately read, the reading will begin at that point in the file.

The File Pointer

- An `EOFException` is thrown when a read causes the file pointer to go beyond the size of the file.
- Writing also takes place at the location pointed to by the file pointer.
- If the file pointer points to the end of the file, data will be written to the end of the file.
- If the file pointer holds the number of a byte within the file, at a location where data is already stored, a write will overwrite the data at that point.

RandomAccessFile methods

- The `RandomAccessFile` class lets you move the file pointer.
- This allows data to be read and written at any byte location in the file.
- The getFilePointer method is used to learn the current file pointer value (long)

  position = rndFile.getFilePointer();

- The seek method is used to move the file pointer.

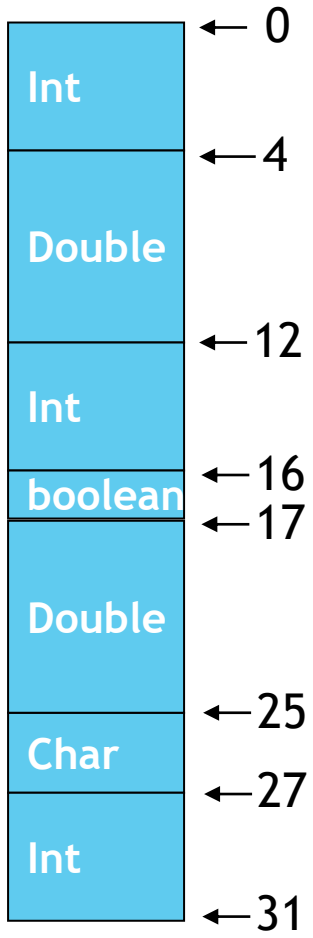
  rndFile.seek(long *position*);

  The argument is the number of the byte that you want to move the file pointer to.

Random Access - example

```java
File newfile = new File("d:\\rand.dat");
RandomAccessFile myfile = new RandomAccessFile(newfile, "rw");

myfile.writeInt(120);
myfile.writeDouble(375.50);
myfile.writeInt('A'+1);
myfile.writeBoolean(true);
myfile.writeChar('X');

// set pointer to the beginning of file and read next two items
myfile.seek(0);
System.out.println(myfile.readInt());
System.out.println(myfile.readDouble());

//set pointer to the 4th item and read it
myfile.seek(16);
System.out.println(myfile.readBoolean());

// set pointer to offset 17 and "append" a double 3.14
myfile.seek(17);
char c = myfile.readChar();
myfile.seek(17);
myfile.writeDouble(3.14);
myfile.writeChar(c);

// Go to the end and "append" an integer 2020
myfile.seek(myfile.length());
myfile.writeInt(2020);

myfile.seek(17);
System.out.println(myfile.readDouble());
System.out.println(myfile.readChar());
System.out.println(myfile.readInt());
System.out.println("File length: "+myfile.length());
myfile.close();
```

Offsets in bytes (at the end of the program)



Program output

```
120
375.5
true
3.14
X
2020
File length: 31
```

A Practical example

Processing a regular text file (UTF-8, not 16) with a fixed record structure

```
// File format : 6235,Kim Blue                    ,1.85,15-04-1992,F
// id     :  4 bytes
// name   : 30 bytes
// cgpa   :  4 bytes
// date   : 10 bytes
// gender :  1 byte
// commas :  4 bytes
// CR LF  :  2 bytes
// TOTAL  : 55 bytes
```

Sample File

```
1234,John Doe                    ,3.16,22-01-1985,M
0021,Jane White                  ,2.05,12-02-1980,F
9821,Tom Jennings                ,1.93,16-03-1990,M
6235,Kim Blue                    ,1.85,15-04-1992,F
1253,Bart Simpson                ,2.01,19-05-1989,M
3451,Alice Wonderland            ,2.37,25-03-1990,F
```

```java
File students = new File("d:\\students.txt");
RandomAccessFile std_file = null;
//Let us display the info of Kim Blue (with student id number 6235) only
try {
    std_file = new RandomAccessFile(students, "rw");
    boolean found = false;

    byte[] id = new byte[4];
    byte[] rec = new byte[55];
    String idno = null;
    String std_rec = null;

    while(std_file.getFilePointer() <= std_file.length()-2) // checking the end of file
    {
        std_file.read(id, 0, 4); //read the first 4 bytes
        idno = new String(id); // covert your byte array to String

        if (idno.equals("6235"))
        {
            std_file.seek(std_file.getFilePointer()-4); // rewind to the beginning of the record
            std_file.read(rec,0,53); // read the complete record excluding the newline at the end
            std_rec = new String(rec);
            found = true;
        }
        if(found) // if the record is found no need to continue reading
            break;

        // Jump to next record (we already read 4 bytes so + 51 for the next record)
        std_file.seek(std_file.getFilePointer() + 51);
    }
    System.out.println(std_rec);
} catch (EOFException e) {
        e.printStackTrace();
} finally
{
    std_file.close();
}
```

```
// File format : 6235,Kim Blue                    ,1.85,15-04-1992,F
// id     :  4 bytes
// name   : 30 bytes
// cgpa   :  4 bytes
// date   : 10 bytes
// gender :  1 byte
// commas :  4 bytes
// CR LF  :  2 bytes
// TOTAL  : 55 bytes
```

Sample File

```
1234,John Doe                    ,3.16,22-01-1985,M
0021,Jane White                  ,2.05,12-02-1980,F
9821,Tom Jennings                ,1.93,16-03-1990,M
6235,Kim Blue                    ,1.85,15-04-1992,F
1253,Bart Simpson                ,2.01,19-05-1989,M
3451,Alice Wonderland            ,2.37,25-03-1990,F
```

Output

```
6235,Kim Blue                    ,1.85,15-04-1992,F
```

```java
File students = new File("d:\\students.txt");
RandomAccessFile std_file = null;

//Let us modify the cgpa of Kim Blue (with student id number 6235) to 3.50
try {
    std_file = new RandomAccessFile(students, "rw");
    boolean found = false;

    byte[] id = new byte[4];
    byte[] rec = new byte[55];
    String idno = null;
    String std_rec = null;

    while(std_file.getFilePointer() <= std_file.length()-2) // checking the end of file
    {
        std_file.read(id, 0, 4); //read the first 4 bytes
        idno = new String(id); // covert your byte array to String

        if (idno.equals("6235"))
        {
            std_file.seek(std_file.getFilePointer()+32); // go to the cgpa field
            std_file.write("3.50".getBytes("UTF-8"));
            std_file.seek(std_file.getFilePointer()-40);
            std_file.read(rec,0,53); // read the complete record excluding the newline at the end
            std_rec = new String(rec);
            found = true;
        }
        if(found) // if the record is found no need to continue reading
            break;

        // Jump to next record (we already read 4 bytes so + 51 for the next record)
        std_file.seek(std_file.getFilePointer() + 51);
    }
    System.out.println(std_rec);
} catch (EOFException e) {
        e.printStackTrace();
} finally {
    std_file.close();
}
```

```
// File format : 6235,Kim Blue                    ,1.85,15-04-1992,F
// id     :  4 bytes
// name   : 30 bytes
// cgpa   :  4 bytes
// date   : 10 bytes
// gender :  1 byte
// commas :  4 bytes
// CR LF  :  2 bytes
// TOTAL  : 55 bytes
```

Output

```
6235,Kim Blue                    ,3.50,15-04-1992,F
```

Sample File(Before)

```
1234,John Doe                    ,3.16,22-01-1985,M
0021,Jane White                  ,2.05,12-02-1980,F
9821,Tom Jennings                ,1.93,16-03-1990,M
6235,Kim Blue                    ,1.85,15-04-1992,F
1253,Bart Simpson                ,2.01,19-05-1989,M
3451,Alice Wonderland            ,2.37,25-03-1990,F
```

Sample File(After)

```
1234,John Doe                    ,3.16,22-01-1985,M
0021,Jane White                  ,2.05,12-02-1980,F
9821,Tom Jennings                ,1.93,16-03-1990,M
6235,Kim Blue                    ,3.50,15-04-1992,F
1253,Bart Simpson                ,2.01,19-05-1989,M
3451,Alice Wonderland            ,2.37,25-03-1990,F
```

Alternative solution

Let's, instead of modifying 4 bytes inside the file;

1. **Find the record**
2. **Read the record into a string**
3. **Modify the string**
4. **Write the new string on top of the old record**

To perform this, we can use RandomAccessFile.readLine() method

!!!!!! readLine() performs dead slow !!!!!!!

INSTEAD ☺

OptimizedRandomAccessFile Class

Available at
https://bitbucket.org/kienerj/io/src/default/

Modified By
Joos Kiener

From Joos Kiener;
" Wrapper of java.io.RandomAccessFile that has an readLine method performing similar to java.io.BufferedReader.readLine while keeping the random access functionality.

java.io.RandomAccessFile.readLine() is very slow as it reads a file byte by byte. This here will perform 2 orders of magnitude faster. For thread-safety all read and write methods are synchronized."

```java
File students = new File("d:\\students.txt");
OptimizedRandomAccessFile std_file = null;
try {
    std_file = new OptimizedRandomAccessFile(students, "rw");
    boolean found = false;
    byte[] id = new byte[4];
    String idno = null;
    String std_rec = null;
    String[] fields;

    while(std_file.getFilePointer() <= std_file.length()-2) // checking the end of file
    {
        std_file.read(id, 0, 4); //read the first 4 bytes
        idno = new String(id); // covert your byte array to String

        if (idno.equals("6235"))
        {
            std_file.seek(std_file.getFilePointer()-4); // Rewind to the beginning of the record
            std_rec = std_file.readLine(); // This is the trick !!!!
            fields = std_rec.split(",");
            fields[2] = "3.45";
            std_rec = String.join(",", fields);
            std_file.seek(std_file.getFilePointer()-55); // Rewind to the beginning of the record
            std_file.write(std_rec.getBytes("UTF-8"));
            found = true;
        }
        if(found) // if the record is found no need to continue reading
            break;

        // Jump to next record (we already read 4 bytes so + 51 for the next record)
        std_file.seek(std_file.getFilePointer() + 51);
    }
    System.out.println(std_rec);
} catch (EOFException e) {
    e.printStackTrace();
} finally
{
    std_file.close();
}
```

Homework #1

- Consider the students file format given in the last example.
- Write a complete Java program which will implement a Student class.
- In the student class you will have the following functionalities.
- Pay attention that the structure of the students.txt file should never be broken.
  1. One class constructor that accepts the full filepath for the students file as its only argument.
  2. A method called modify(stdid, field, new_value) which will modify the requested field to the provided new value of the given student. You should consider finding the student only by reading the student ids and stopping once you find.
  3. An insert(stdid,name,cgpa,date,gender) method which will append a student to the end of the student file.
  4. A delete(stdid) method which will delete a student record (line) completely.
  5. display() which will display all students on the monitor (tab separated).
  6. A method called stats() which will create a new file called stats.txt in the same path of the students.txt which will contain how many female and male students there are, the average cgpa and the total student count on separate files.
- How the student class will be used is your experiment which will be explained and tested during the lab hours.